Part B: Big Data Foundations and Architecture – MetroFlow

The proposed MetroFlow architecture should support three main requirements:

1. Real-time operations: Process GPS and traffic data to generate alerts about delays and congestion within 30 seconds.

2. Historical analytics: Store and process large volumes of data for daily and monthly reports covering routes, passenger demand, reliability, and maintenance.

3. Security and scalability: Retain raw data for five years, accommodate increasing data volumes, and protect passenger identifiers and vehicle location information.

B1. Analyse the five Vs

The five Vs describe the main characteristics of big data and explain why MetroFlow requires a scalable data architecture.

Big Data V	Scenario-specific example	Effects on design system
Volume	MetroFlow operates 2500 buses, each generating a GPS event every five (5) seconds. This could produce about 500 GPS events per second, assuming all buses are active. In addition, ticketing systems generate 3 million passenger tap records daily, while cameras produce large video files	The system requires scalable storage such as object cloud storage to handle the amount of data (increasing data volume). Distributed processing is also necessary to analyse large datasets efficiently.
Velocity	GPS events are generated every five seconds, while passenger taps and sensor readings arrive continuously. The control room requires congestion and delay alerts within 30 seconds.	A streaming platform such as Apache Kafka should collect incoming events and make them available for near real-time processing.
Variety	MetroFlow collects structured ticketing records, GPS coordinates, road camera videos, JSON logs from maintenance systems, and sensor readings.	The architecture should support various data formats. A data lake using cloud object storage can retain raw data, while structured datasets can be prepared for reporting and analytics.

Veracity	There may be inaccuracies in the GPS readings as a result of poor network coverage, faults from sensors or incorrect timestamps. Maintenance logs may also contain missing or inconsistent values.	Before information is used for alerts and reports, it is required to improve data quality through data validation, cleaning, timestamp checks, and duplicate detection.
Value	Analysing passenger demand, route delays, congestion, and vehicle maintenance can help MetroFlow improve scheduling, operational efficiency, and reliability.	Analytics tools and reporting systems should transform collected data into useful information for control-room operators and transport planners.

In conclusion, MetroFlow portrays all five characteristics (Big Vs) of big data. Its architecture must therefore support high-volume ingestion, rapid processing, multiple data formats, reliable data quality, and meaningful analytics.

B2. Explain conventional system limitations

The three likely limitations of relying on one conventional database server for the full workload are;

1. **Limited storage capacity:** In the MetroFlow scenario, a single database server might struggle to store five years of GPS events. Passenger records, video files and maintenance data. The increase in buses will require storage to continue to increase.
2. **Processing and performance bottlenecks:** The processing of approximately 5oo GPS events per second, with millions of ticketing records and video data, could overwhelm one server, which may affect the ability to generate alerts within the required timeframe.
3. **Limited scalability and availability:** In the MetroFlow scenario, a single server has limited resources and can become a single point of failure. As such, both real-time monitoring and historical reporting could be disrupted if it becomes unavailable.

Scale-up (Vertical scaling) means increasing the resources of one server, such as adding more RAM, CPU power or storage capacity. With scaling up, the existing machine is scaled up by improving performance. However, it is limited by the maximum hardware capacity. Whilst

Scale-out (Horizontal scaling) could be defined or explained as adding more processing nodes or servers to distribute storage and processing workloads across multiple machines. Scale-out allows the system to handle increasing workloads by expanding the cluster.

Recommendation on Approach

I recommend MetroFlow adopt a scale-out architecture, supported by cloud-based storage and distributed processing.

Due to the fact that MetroFlow generates data continuously from 2500 buses, with GPS events arriving every five seconds and approximately three million passenger taps records generated daily, this approach is more suitable. The increase in data volume as a result of new vehicles being introduced will be difficult to handle if relying on a single server.

A distributed architecture lets MetroFlow add processing nodes and more storage as demand grows. For example, Apache Kafka can split GPS and ticketing events into many partitions, and a distributed engine like Apache Spark can process large datasets using many workers.

Cloud object storage can keep raw GPS records, passenger transactions, videos and maintenance logs for five years without putting all of that load on one database server. Compute resources can also be scaled separately from storage so MetroFlow can add processing power when it needs to generate reports or analyse large data sets.

Although a scale-out architecture brings management and coordination work, it gives better scalability, spread of workload and resilience. Therefore, it is a long-term solution for MetroFlow's growing data needs.

B3. Choose the processing approach (6 marks)

MetroFlow has different processing requirements depending on how quickly the information must be available and how it will be used.

Classification of workloads

Workload	Processing approach	Processing approach

Delay and congestion alerts	Streaming	GPS events and road sensor readings must be processed continuously so that the control room receives alerts within 30 seconds.
Daily route reports	Batch	Throughout the day at scheduled intervals, route performance, passenger demand, reliability, and maintenance records can be collected throughout the day and processed together
Long-term planning	Batch	Monthly and historical reports require analysing large volumes of retained data to identify trends and support future transport planning.

How Apache Kafka fits in the architecture

Apache Kafka will collect data from MetroFlow's GPS devices, ticketing systems, traffic sensors, and maintenance systems. The data will be organised into topics such as gps-events, ticket-events, and maintenance-events.

Each topic can be divided into partitions to allow data to be processed in parallel as the volume grows. Streaming tools such as Apache Flink can use the incoming GPS and traffic data to detect delays and congestion and send alerts to the control room within 30 seconds.

At the same time, the data can be stored in cloud object storage for long-term retention and later analysed using Apache Spark to produce daily and monthly reports.

B4. Draw a preliminary architecture

Explanation of the architecture

MetroFlow architecture starts with GPS devices, ticketing systems, road cameras and maintenance systems that generate data. Apache Kafka collects and distributes continuous events while video ingestion moves camera footage into cloud object storage. The cloud data lake keeps data for five years, letting storage capacity grow independently of computing resources.

For real-time operations, Apache Flink or Spark Structured Streaming handles GPS and traffic events to spot congestion and delays, sending alerts to the control room within 30 seconds. For analysis, Apache Spark does distributed batch processing on the stored data. The processed datasets are organised in a lakehouse and made available to an analytics or data warehouse layer. Finally, reporting tools give monthly insights into passenger demand, route performance, reliability and vehicle maintenance.

B5 Apply responsible design

MetroFlow handles sensitive passenger and vehicle information; as such, privacy, security, and accountability must be included in the architecture.

1. **Privacy risk: Exposure of passenger identifiers and vehicle location data.**
   This risk implies that unauthorised access to passenger records could reveal personal travel records of passengers, while exposed GPS data could reveal the real-time locations and movement of buses.
2. **Governance control: Data encryption, pseudonymisation, and audit logging.**
   In cases where direct identification is unnecessary, passenger identifiers should be pseudonymised. Also, data should be encrypted both in transit and at rest. Access and data usage should be recorded in audit logs.
3. **Role-based access control (RBAC): Assigning permissions based on job responsibilities.**
   Control room operators should be able to access the vehicle locations and operational information needed for real-time monitoring. Administrators should have restricted, monitored access to sensitive datasets.

Traceability and auditing

The architecture needs to keep track of who looked at the data, when they looked and what they did. Every set of data should also have information about where it came from, when it was collected, how it was processed and what changes were made to it.

This makes it possible for MetroFlow to follow data from the beginning all the way through when it is brought in, processed, stored and shared. It also makes it easier to find if someone accessed the data without permission, look into issues with the data quality and show that the rules for protecting data are being followed, including GDPR.

Overall conclusion

MetroFlow should use a data architecture that can grow and works in the cloud. It should use Apache Kafka to bring in events, cloud object storage to keep data for five years and a system that can process data across many computers using Apache Spark or Apache Flink.

Real-time processing will help the control room get alerts every 30 seconds, while older data will be processed in batches for monthly reports and for planning transportation over a longer time.

In the end, using encryption, making data anonymous, controlling who can access it and keeping records of all access will help make sure that all the data about passengers and cars is handled in a responsible way.

Assumptions made

1. **The 500 GPS events per second assumes all 2500 buses are active simultaneously**
2. **MetroFlow does not need to deploy both Spark and Flink, as they are both alternatives for distributed processing.**